

TECHNICAL SPECIFICATIONS

DeepEcho Faster-Whisper

Specifications · v1.1.0-dev

Complete functional and technical reference for the local Faster-Whisper transcription system: installation, models, audio processing, file output, validation and packaging.

Author / Creator: Bruno DELNOZ / @NoXoZ.be

Contact: bruno.delnoz@protonmail.com

Document: SPECIFICATIONS.md → SPECIFICATIONS.pdf

Version: v1.1.0-dev

Document date: 2026-10-09 18:32 CEST

Status: Validation candidate; not a release tag.

TECHNICAL DOCUMENT · VALIDATION CANDIDATE

Contents

Interactive table of contents · 135 numbered sections · click any title to navigate · PDF sidebar bookmarks included.

Seven thematic groups · each group and numbered section is clickable.

Project foundations	001-012	Model management	026-041
001 Document status	5	026 Model-management architecture	13
002 Project purpose	5	027 Default model storage	13
003 Primary design principles	5	028 Model format	13
004 Repository source set	6	029 Model completeness state	14
005 Runtime-only repository-local paths	6	030 Model list action	14
006 Runtime media-directory paths	7	031 Static model reference in help	14
007 Versioning policy for this candidate	7	032 Single-model download	15
008 Metadata policy	7	033 Multi-model download	15
009 Author metadata	8	034 Duplicate requested model names	15
010 Changelog policy	8	035 Existing complete model behavior	15
011 Normal user interfaces	8	036 Incomplete/corrupt model behavior	15
012 Shared SOLO-style control conventions	8	037 Model --force	16
		038 Model force simulation	16
		039 Invalid model names in a batch	16
		040 Model manager no-argument behavior	16
		041 Model manager control options	16
Installation & Git hygiene	013-025		
013 Installation architecture	9		
014 install_pip.sh responsibilities	9		
015 install_pip.sh no-action semantics	10		
016 install_pip.sh real-action logging	10		
017 install_pip.sh purge scope	10		
018 requirements.txt ownership	10		
019 install.sh responsibilities	11		
020 install.sh log naming	11		
021 Installer log Git policy	12		
022 .gitignore preservation invariant	12		
023 Required .gitignore protection set	12		
024 Existing .gitignore duplicates	12		
025 Installer simulation behavior	13		

Transcription pipeline	042-066	Files, timestamps & logging	067-094
042 Transcription architecture	17	067 Timestamped Markdown behavior	22
043 Transcription no-argument behavior	17	068 Generated file timestamp format	22
044 Transcription control interface	17	069 Single run timestamp invariant	23
045 Default language	17	070 Timestamp collision avoidance	23
046 Default device	18	071 Transcription --force	23
047 Default compute type	18	072 Default timestamped Markdown location	23
048 Explicit model requirement	18	073 Default plain-transcript location	24
049 Source directory default	18	074 Default transcription log location	24
050 Default source pattern	19	075 Multi-source same-directory logging	24
051 Supported source selection forms	19	076 Multi-directory logging	24
052 Shell-expanded glob regression requirement	19	077 Multi-directory plain outputs	25
053 Filenames with spaces regression requirement	19	078 Explicit destination behavior	25
054 Python-backend source parity	20	079 Explicit destination collision detection	25
055 Primary media format	20	080 Output directory creation	25
056 Source protection	20	081 Transcription output formats	25
057 Raw transcription policy	20	082 SRT status	26
058 VAD policy	20	083 WebVTT status	26
059 VAD rationale constraint	21	084 JSON status	26
060 Normalization policy	21	085 Speaker diarization status	26
061 Amplification factor	21	086 Future diarization target	26
062 Amplification dB	21	087 Prerequisite action for transcription	26
063 Amplification exclusivity	21	088 Transcription simulation semantics	27
064 FFmpeg requirement	22	089 Transcription error handling	27
065 PyAV/Faster-Whisper compatibility issue	22	090 Transcription logging content	27
066 PyAV compatibility requirement	22	091 Installer logging content	28
		092 Repository log isolation	28
		093 Privacy/data separation	28
		094 Git model incident prevention	28

Validation & documentation

095-115

095	ZIP/package policy	29
096	ZIP root layout	29
097	ZIP integrity validation	29
098	Shell syntax validation	29
099	Python syntax validation	29
100	No-argument validation	29
101	Help validation	29
102	Changelog validation	30
103	Model parser validation	30
104	Transcription source parser validation	30
105	PyAV regression validation	30
106	Output-layout validation	31
107	Installer-log validation	31
108	.gitignore mutation validation	31
109	.gitignore runtime idempotence	31
110	Requirements validation	32
111	Documentation requirements	32
112	EXAMPLES.md completeness	32
113	INSTALL.md automation contract	32
114	README.md current-state contract	32
115	CHANGELOG.md project history	32

Compatibility & final acceptance

116-135

116	Current runtime baseline	33
117	Python minimum	33
118	Network/package behavior	33
119	Model aliases	33
120	English-only model naming note	33
121	Transcription output extension preservation	34
122	Source timestamp units	34
123	Runtime source ordering	34
124	Duplicate source suppression	34
125	Symlink/path resolution	34
126	.venv interpreter discovery	35
127	Backend discovery	35
128	Git operations boundary	35
129	Secrets policy	35
130	Project-package exclusion policy	35
131	Requirements file in package	36
132	Package completeness	36
133	Validation before release	36
134	Current acceptance checklist	36
135	Changelog	37

DeepEcho_faster_whisper — Specifications

1. Document status

This document is the detailed functional and technical contract for the current `DeepEcho_faster_whisper` validation candidate.

The current candidate is identified as:

v1.1.0-dev

This is a major project update but **not yet a final release declaration**. The user must validate the delivered behavior before any release/tag decision.

The specification intentionally records implemented behavior and agreed constraints in detail so that future changes can be checked for regressions.

2. Project purpose

`DeepEcho_faster_whisper` provides a reusable local Faster-Whisper transcription workflow with four distinct responsibilities:

1. bootstrap the isolated Python environment;
2. install and validate the Faster-Whisper runtime;
3. manage local Faster-Whisper models;
4. transcribe private media without coupling the media location to the public repository.

The repository contains reusable code and documentation. Private media, downloaded model payloads, generated transcripts and generated operational logs are runtime data and are not repository source.

3. Primary design principles

The project SHALL preserve the following design principles:

- the shell scripts are the normal user-facing interfaces;
- Python scripts are the backends where Python functionality is appropriate;
- system-wide pip installation is forbidden;
- the Python runtime is project-local under `.venv/`;
- source media SHALL NOT be modified;
- source media SHALL NOT need to be copied into the repository;
- generated transcripts SHALL be source-oriented rather than repository-oriented;
- generated runtime data SHALL be protected from accidental Git commits;
- installation actions SHALL be explicit;

- no-argument script invocation SHALL display help and perform no business action;
- prerequisite checks SHALL be read-only;
- simulation actions SHALL not perform the corresponding business action;
- model downloads SHALL be explicit;
- transcription SHALL not silently download a missing model;
- VAD, normalization and amplification SHALL never be enabled silently;
- raw spoken content SHALL not be deliberately censored, summarized or editorially rewritten;
- speaker identities SHALL never be fabricated;
- every code/document modification SHALL carry an incremented version/date/changelog where the file supports version metadata.

4. Repository source set

The validation package SHALL contain the following repository files:

```
.gitignore
requirements.txt
install_pip.sh
install.sh
getModels.sh
getModels.py
transcribe.sh
transcribe.py
README.md
CHANGELOG.md
INSTALL.md
SPECIFICATIONS.md
EXAMPLES.md
```

WHY.md SHALL NOT be part of this project version.

5. Runtime-only repository-local paths

The following paths MAY exist locally but SHALL NOT be considered repository source:

```
.venv/
.VENV/
venv/
models/
logs/
.logs/
.zip/
.old/
.exports/
.export/
.transcription/
__pycache__/
```

This list is intentionally protective. Some directories are normally created outside the repository, but if they appear below the repository they SHALL still be excluded from Git.

6. Runtime media-directory paths

A media directory MAY contain source media and generated runtime data such as:

```
source.mp4
source.mp4.transcription_timestamps-YYYYMMDD-HHMM-SS.md
.transcription/
.logs/
```

These files/directories are private working data rather than public project source.

7. Versioning policy for this candidate

All modified scripts and project documents SHALL be incremented for this validation candidate.

The version string used by this package is:

```
V1.1.0-dev
```

or lower-case Markdown equivalent:

```
v1.1.0-dev
```

The `-dev` suffix explicitly indicates that this package requires user validation and is not yet a release/tag commitment.

8. Metadata policy

Scripts SHALL include metadata covering at least:

- script name;
- author;
- email;
- version;
- date/time;
- target usage;
- internal changelog.

Project Markdown documents SHALL include document metadata covering at least:

- document name;
- author;
- email;
- version;
- date/time;
- project;

- status;
- short description.

9. Author metadata

Project author metadata is:

```
Author: Bruno DELNOZ  
Email: bruno.delnoz@protonmail.com
```

10. Changelog policy

Changelogs are append-only histories.

A new modification SHALL add a new entry while preserving older entries.

Project-level history belongs in `CHANGELOG.md`.

Script-specific implementation history SHALL also be present through each script's `--changelog` behavior or internal changelog section.

11. Normal user interfaces

The canonical user-facing scripts are:

```
install_pip.sh  
install.sh  
getModels.sh  
transcribe.sh
```

The Python backends are:

```
getModels.py  
transcribe.py
```

Normal documentation SHALL prefer the shell wrappers for ordinary user workflow while backend examples MAY be documented for debugging/testing.

12. Shared SOLO-style control conventions

Where applicable, scripts SHALL use these control forms:

```
--help      -h  
--exec      -exe  
--simulate  -s  
--prerequis -pr  
--changelog -ch
```


Installer scripts additionally SHALL expose:

```
--install    -i  
--purge      -pu
```

No argument SHALL display help and perform no installation, download or transcription.

13. Installation architecture

Installation SHALL be split into two physical stages:

```
1. install_pip.sh  
2. install.sh
```

`install_pip.sh` owns Python bootstrap and `requirements.txt` preparation.

`install.sh` owns Faster-Whisper dependency installation and runtime validation.

14. install_pip.sh responsibilities

`install_pip.sh` SHALL:

- display help with no arguments;
- implement `--help/-h` ;
- implement `--prerequis/-pr` ;
- implement `--simulate/-s` ;
- implement `--install/-i` ;
- implement `--exec/-exe` as the real bootstrap equivalent;
- implement `--changelog/-ch` ;
- implement `--purge/-pu` ;
- require Python ≥ 3.9 ;
- verify Python `venv` capability;
- check required shell utilities;
- check free disk space;
- require at least 4096 MiB free on the current working filesystem;
- check the project filesystem separately when required;
- create a project-local `.venv` when absent;
- preserve an existing valid `.venv` ;
- upgrade pip/setuptools/wheel only inside `.venv` ;
- use `--no-cache-dir` for pip bootstrap;
- create `requirements.txt` if absent;
- preserve existing requirements content;
- ensure a Faster-Whisper requirement is present;
- ensure repository `.gitignore` runtime protection during real install actions;

- create timestamped installer logs during real modifying actions;
- never install project packages system-wide.

15. `install_pip.sh` no-action semantics

The following SHALL remain non-modifying:

```
no arguments
--help
--prerequis
--simulate
--changelog
```

In particular, these actions SHALL NOT create the installer `logs/` directory merely to record a read-only operation.

16. `install_pip.sh` real-action logging

Real bootstrap and purge actions SHALL create logs under:

```
<repo>/logs/
```

Naming SHALL follow:

```
install_pip-VERSION-YYYYMMDD-HHMM-SS.log
```

Seconds SHALL be explicit in the filename.

17. `install_pip.sh` purge scope

`--purge` SHALL remove only the project-local `.venv` runtime.

It SHALL preserve at least:

- `requirements.txt`;
- downloaded models;
- source/documentation;
- installer logs;
- `.gitignore`.

18. `requirements.txt` ownership

`install_pip.sh` owns creation/preservation of `requirements.txt`.

If absent, it SHALL create a file declaring:

```
faster-whisper==1.2.1
```

If present, it SHALL preserve existing content and append the declaration only when no Faster-Whisper requirement exists.

`install.sh` SHALL consume this file rather than replacing it.

19. `install.sh` responsibilities

`install.sh` SHALL:

- display help with no arguments;
- implement `--help/-h`;
- implement `--prerequis/-pr`;
- implement `--simulate/-s`;
- implement `--install/-i`;
- implement `--exec/-exe` as equivalent real installation action;
- implement `--changelog/-ch`;
- implement `--purge/-pu`;
- require a valid project-local `.venv`;
- require a valid `requirements.txt`;
- verify Faster-Whisper is declared;
- check free disk space;
- ensure `.gitignore` protection during real actions;
- install packages strictly through `.venv/bin/python -m pip`;
- use `--no-cache-dir`;
- run `pip check`;
- validate Faster-Whisper import/version;
- validate CTranslate2 import/version;
- validate PyAV import/version;
- validate `WhisperModel` import;
- never download Whisper models;
- create timestamped logs under repository `./logs/` for real modifying actions.

20. `install.sh` log naming

Runtime-install logs SHALL use:

```
install-VERSION-YYYYMMDD-HHMM-SS.log
```

under:

```
<repo>/logs/
```

21. Installer log Git policy

`logs/` and `*.log` SHALL be excluded from Git.

The installer SHALL ensure these rules exist without deleting existing `.gitignore` content.

22. `.gitignore` preservation invariant

`.gitignore` is special.

The project SHALL obey these rules:

- existing lines SHALL never be removed automatically;
- existing comments SHALL never be removed automatically;
- existing lines SHALL never be reordered automatically;
- duplicate existing entries SHALL never be deduplicated automatically;
- required entries missing from the file MAY be appended;
- a missing `.gitignore` MAY be created by a real installation action;
- the delivered package SHALL preserve all historical `.gitignore` lines from the provided baseline.

23. Required `.gitignore` protection set

Installer logic SHALL ensure the following exact protective entries are present or appended:

```
.venv/  
.VENV/  
venv/  
models/  
.zip/  
.old/  
.logs/  
logs/  
.exports/  
.export/  
.transcription/  
__pycache__/  
*.pyc  
*.log
```

The directory form is `.logs/`; `.log/` SHALL NOT be introduced as a directory rule.

24. Existing `.gitignore` duplicates

If `.gitignore` already contains duplicate entries such as repeated `*.log`, `.old/` or `.zip/`, those duplicates SHALL remain untouched.

The project SHALL NOT perform cleanup simply for aesthetics.

25. Installer simulation behavior

Installer simulation SHALL describe:

- runtime checks;
- `.gitignore` additions that a real run would ensure;
- `.venv` creation/update;
- requirements handling;
- package installation/validation;
- log location that a real run would use.

Simulation SHALL NOT create these artifacts.

26. Model-management architecture

Model management SHALL be separated from general installation.

The user-facing interface is:

```
getModels.sh
```

The backend is:

```
getModels.py
```

The installer SHALL NOT hide model downloads inside installation.

27. Default model storage

Default model storage SHALL be:

```
<repo>/models/<model-name>/
```

`models/` SHALL be Git-ignored.

The user MAY override model storage with `--models-dir`.

28. Model format

Faster-Whisper/CTranslate2 models SHALL be treated as directories containing model assets, not as a single GGML/GGUF model file.

Typical completeness indicators include at least:

```
config.json  
model.bin  
tokenizer.json
```

29. Model completeness state

The model manager SHALL distinguish:

```
INSTALLED
INCOMPLETE
not installed
```

A directory SHALL NOT be trusted merely because its name exists.

30. Model list action

The model manager SHALL support:

```
./getModels.sh --exec --list
```

This list SHALL use the installed Faster-Whisper runtime registry as the authoritative live source and SHALL show local status for each name.

No separate `--local` action is required.

31. Static model reference in help

The help output for `getModels.sh` and `getModels.py` SHALL include a complete static model reference **immediately before the EXAMPLES section**.

The required reference list is:

```
1. tiny.en
2. tiny
3. base.en
4. base
5. small.en
6. small
7. medium.en
8. medium
9. large-v1
10. large-v2
11. large-v3
12. large
13. distil-large-v2
14. distil-medium.en
15. distil-small.en
16. distil-large-v3
17. distil-large-v3.5
18. large-v3-turbo
19. turbo
```

The static reference is documentation convenience. `--exec --list` remains the live runtime view.

32. Single-model download

The model manager SHALL support:

```
./getModels.sh --exec --download --model tiny
```

A missing valid model SHALL be downloaded explicitly.

33. Multi-model download

The model manager SHALL accept several model names after a single `--model` option.

Required syntax example:

```
./getModels.sh --exec --download --model base small medium
```

The Python backend SHALL accept the same form.

34. Duplicate requested model names

When the same model name is supplied multiple times in one invocation, the backend SHOULD process the distinct requested names once while preserving first-seen order.

35. Existing complete model behavior

Without `--force`, a complete local requested model SHALL be:

- detected;
- reported;
- skipped;
- left untouched.

Other requested models SHALL continue processing.

36. Incomplete/corrupt model behavior

Without `--force`, an existing but incomplete local model directory SHALL:

- be reported as incomplete/corrupt;
- not be silently trusted;
- not be silently deleted;
- cause a non-zero aggregate result;
- not prevent independent valid requested models from being processed.

The user SHALL be directed to use `--force` when replacement is intended.

37. Model --force

The model manager SHALL implement:

```
--force
```

only with `--download`.

`--force` SHALL apply to every requested valid model in the invocation.

For an existing requested target, `--force` SHALL remove the local target and redownload it.

This behavior is explicitly intended for suspected corruption or deliberate replacement.

38. Model force simulation

The following SHALL be non-destructive:

```
./getModels.sh --simulate --download --model base small medium --force
```

It SHALL describe that existing targets would be removed/redownloaded without actually changing files.

39. Invalid model names in a batch

An invalid model name SHALL NOT be silently ignored.

The program SHALL:

- report the invalid name clearly;
- continue processing other valid requested names where practical;
- return a non-zero aggregate status when one or more requested names are invalid/failed.

40. Model manager no-argument behavior

Launching `getModels.sh` or `getModels.py` without arguments SHALL display help and perform no model download.

41. Model manager control options

The model manager SHALL support:

```
--help/-h  
--exec/-exe  
--simulate/-s  
--prerequis/-pr  
--changelog/-ch
```

Business actions SHALL include:


```
--list  
--download
```

Model options SHALL include:

```
--model NAME [NAME ...]  
--models-dir PATH  
--force
```

42. Transcription architecture

The normal transcription interface SHALL be:

```
transcribe.sh
```

The backend SHALL be:

```
transcribe.py
```

The shell wrapper SHALL validate its CLI sufficiently to produce immediate user-friendly errors, then forward the original supported business arguments to the Python backend.

43. Transcription no-argument behavior

No-argument execution SHALL display complete help and perform no transcription or filesystem output creation.

44. Transcription control interface

The transcription layer SHALL support:

```
--help/-h  
--exec/-exe  
--simulate/-s  
--prerequis/-pr  
--changeLog/-ch
```

It SHALL NOT require installer-specific `--install` or `--purge` actions.

45. Default language

Default transcription language SHALL be:

```
fr
```

The user SHALL NOT need to supply `--language fr` for normal French transcription.

`--language auto` SHALL request automatic language detection.

Other explicit language codes MAY be supplied.

46. Default device

Default CTranslate2 device SHALL be:

```
cpu
```

The device SHALL remain configurable through:

```
--device
```

47. Default compute type

Current candidate default SHALL be:

```
int8
```

The compute type SHALL remain configurable through:

```
--compute-type
```

This candidate default MAY be revisited after real performance/quality validation; it SHALL not be silently changed without versioning/documentation.

48. Explicit model requirement

Transcription execution/simulation SHALL require:

```
--model <NAME>
```

The model SHALL already exist locally under the selected models directory.

A missing model SHALL generate an error directing the user to `getModels.sh`.

Transcription SHALL NOT auto-download models.

49. Source directory default

Default source base SHALL be:

Relative source patterns SHALL resolve against the current/source directory.

50. Default source pattern

During transcription execution/simulation, when `--source` is omitted, the default pattern SHALL be:

```
*.mp4
```

51. Supported source selection forms

The transcription layer SHALL support:

- one explicit file;
- multiple explicit files;
- repeated `--source` arguments;
- multiple source values after one `--source` ;
- quoted glob patterns;
- unquoted shell-expanded glob patterns;
- filenames containing spaces;
- absolute source paths;
- relative source paths;
- `--source-dir` base override.

52. Shell-expanded glob regression requirement

The following command SHALL work when several MP4 files exist:

```
./transcribe.sh --simulate --model tiny --source *.mp4
```

If Bash expands `*.mp4` into several filenames, every expanded filename SHALL remain part of the source group rather than becoming an unknown argument.

53. Filenames with spaces regression requirement

A source such as:

```
2015-02-06 07.01.33.mp4
```

SHALL work when quoted directly and SHALL remain one argument when produced by shell glob expansion.

54. Python-backend source parity

`transcribe.py` SHALL independently accept shell-expanded multiple values after `--source`, not only when called through the shell wrapper.

This guarantees that the same glob/space fix exists in both `.sh` and `.py` paths.

55. Primary media format

MP4 SHALL be a primary source format.

The backend MAY additionally accept common video/audio formats including:

```
.avi .mkv .mov .wmv .flv .webm .m4v .3gp .ogv .ts .mts .m2ts  
.mp3 .wav .flac .m4a .aac .ogg .opus
```

Unsupported extensions SHALL not be treated as valid media sources.

56. Source protection

The original source media SHALL never be modified by transcription.

Any normalization/amplification SHALL operate on a temporary copy.

Temporary preprocessing files SHALL be removed with the temporary workspace after the source is processed.

57. Raw transcription policy

Transcription output SHALL reflect raw model output as faithfully as practical.

The program SHALL NOT deliberately:

- censor vocabulary;
- sanitize offensive content;
- summarize spoken material;
- rewrite statements into more polite language;
- remove content because it is uncomfortable;
- invent content absent from the model output;
- fabricate speakers.

The current writer strips surrounding whitespace from each model segment and joins preserved segment text into output lines.

58. VAD policy

VAD SHALL be OFF by default.

The user MAY enable it explicitly with:

```
--vad
```

The user MAY explicitly select the default OFF state with:

```
--no-vad
```

No hidden VAD SHALL be applied.

59. VAD rationale constraint

VAD distinguishes speech-like regions from silence/noise; it is not a simple volume control.

Because weak/quiet speech may be important evidence, the project SHALL keep VAD disabled by default unless the user opts in.

60. Normalization policy

Normalization SHALL be OFF by default.

`--normalize` SHALL explicitly request FFmpeg preprocessing with a temporary audio copy.

The current implementation uses `loudnorm`.

61. Amplification factor

`--amplify <FACTOR>` SHALL explicitly request temporary volume multiplication.

The factor SHALL be positive.

Example:

```
--amplify 2
```

62. Amplification dB

`--amplify-db <DB>` SHALL explicitly request temporary dB volume adjustment.

Example:

```
--amplify-db 6
```

63. Amplification exclusivity

`--amplify` and `--amplify-db` SHALL be mutually exclusive.

The script SHALL reject simultaneous use.

64. FFmpeg requirement

FFmpeg SHALL only be mandatory when explicit preprocessing is requested.

A transcription that uses no normalization/amplification SHALL not fail merely because the system FFmpeg executable is absent.

65. PyAV/Faster-Whisper compatibility issue

The validated environment includes:

```
faster-whisper 1.2.1
PyAV 19.0.1
```

Faster-Whisper 1.2.1 uses `av.open(..., metadata_errors=...)` while PyAV 19 removed that keyword.

The observed failure was:

```
open() got an unexpected keyword argument 'metadata_errors'
```

66. PyAV compatibility requirement

`transcribe.py` SHALL detect PyAV major version 19 or newer and install a process-local compatibility wrapper that removes only the obsolete `metadata_errors` keyword before delegating to the actual PyAV `open()` implementation.

The project SHALL NOT require a PyAV downgrade solely for this incompatibility.

The project SHALL NOT modify installed Faster-Whisper package source files on disk to implement this workaround.

67. Timestamped Markdown behavior

Timestamped Markdown SHALL be ON by default.

It SHALL be controllable with:

```
--timestamp
--no-timestamp
```

With timestamps enabled, each segment line SHALL use a form equivalent to:

```
[HH:MM:SS.mmm -> HH:MM:SS.mmm] text
```

68. Generated file timestamp format

Every generated transcript/log filename SHALL contain a run timestamp ending in:

```
YYYYMMDD-HHMM-SS
```

Example:

```
20261009-1832-45
```

The separator before seconds is required so seconds are visually explicit.

69. Single run timestamp invariant

One transcription invocation SHALL allocate one run timestamp.

That exact timestamp SHALL be reused by:

- timestamped Markdown outputs;
- plain Markdown outputs;
- plain TXT outputs;
- transcription logs across all involved source directories.

This creates a direct visible association between artifacts from the same run.

70. Timestamp collision avoidance

The timestamp design exists primarily to avoid overwrites.

Normal execution SHALL check timestamped candidate paths.

When a collision exists and `--force` is not selected, the backend SHALL choose a subsequent free second rather than silently overwrite an earlier run.

71. Transcription `--force`

The transcription CLI MAY retain `--force` as an explicit escape hatch for an exact timestamped target collision.

Because normal runs allocate unique timestamps, ordinary operation SHALL not require `--force`.

72. Default timestamped Markdown location

For source:

```
/path/source.mp4
```

timestamped Markdown SHALL remain in the same directory as the source:

```
/path/source.mp4.transcription_timestamps-YYYYMMDD-HHMM-SS.md
```

It SHALL NOT be moved into `.transcription/` by default.

73. Default plain-transcript location

Plain Markdown and plain TXT SHALL go to a hidden subdirectory named:

```
.transcription/
```

inside the source directory by default.

Example:

```
/path/.transcription/source.mp4.transcription-YYYYMMDD-HHMM-SS.md  
/path/.transcription/source.mp4.transcript-YYYYMMDD-HHMM-SS.txt
```

74. Default transcription log location

Transcription runtime logs SHALL go to:

```
.logs/
```

under the source directory.

Example:

```
/path/.logs/transcribe-VERSION-YYYYMMDD-HHMM-SS.log
```

The directory name SHALL be `.logs/`.

75. Multi-source same-directory logging

When several sources from the same directory are processed in one invocation, that directory SHALL receive one run log for the invocation.

The log SHALL contain run-level and per-source information.

76. Multi-directory logging

When one invocation processes sources from multiple directories, every unique source directory SHALL receive its own copy of the run log under its local `.logs/` directory.

All these log files SHALL use the same run timestamp.

This ensures that each evidence/media directory retains the operational log relevant to the run that touched its sources.

77. Multi-directory plain outputs

Without `--dest-dir`, every source SHALL use its own source-local `.transcription/` directory.

A batch SHALL therefore preserve locality even when sources come from several directories.

78. Explicit destination behavior

`--dest-dir <PATH>` SHALL override the base directory for plain Markdown/TXT outputs only.

Plain outputs SHALL go under:

```
<dest-dir>/transcription/
```

Timestamped Markdown SHALL remain source-local.

Transcription logs SHALL remain source-local.

This split preserves the user's explicit requirement that timestamped evidence-style Markdown stay beside the media source.

79. Explicit destination collision detection

If several sources with identical basenames are directed to one common `--dest-dir` and would resolve to identical output paths for the same run timestamp, the program SHALL refuse the ambiguous collision rather than overwrite one source's transcript with another.

80. Output directory creation

Real execution SHALL automatically create required output directories such as:

```
.transcription/  
.logs/
```

The user SHALL NOT need to create them manually.

Simulation SHALL NOT create them.

81. Transcription output formats

The current candidate SHALL generate:

- timestamped Markdown when timestamps are enabled;
- plain Markdown;
- plain TXT.

82. SRT status

SRT generation is not implemented in this candidate.

It remains a possible future extension.

83. WebVTT status

WebVTT generation is not implemented in this candidate.

WebVTT refers to the `.vtt` subtitle/text-track format.

84. JSON status

JSON transcription output is not required and is not generated in this candidate.

85. Speaker diarization status

Speaker diarization is not implemented in this candidate.

Faster-Whisper transcription alone SHALL NOT be presented as speaker diarization.

The program SHALL NOT invent labels such as `Speaker 1`, `Speaker 2`, `Bruno` or another person's name without a real diarization/identification layer.

86. Future diarization target

A future diarization extension MAY target output resembling:

```
Speaker 1 [time]: text
Speaker 2 [time]: text
```

and MAY later support mapping stable speaker clusters to known names when the necessary evidence/tooling exists.

This is future scope, not current behavior.

87. Prerequisite action for transcription

`--prerequis` SHALL validate the local runtime without modifying it.

Checks SHALL cover at least:

- Python;
- Faster-Whisper package;
- CTranslate2;
- PyAV;
- PyAV compatibility state;
- FFmpeg availability as optional/conditional information;

- local models directory;
- detected local complete models;
- free space.

88. Transcription simulation semantics

`--simulate` SHALL:

- resolve sources;
- resolve model path;
- validate required runtime options;
- allocate/display a run timestamp candidate;
- display planned output paths;
- display planned log directories;
- perform no transcription;
- create no output directories;
- create no logs;
- modify no source media.

89. Transcription error handling

Failures SHALL return non-zero status.

Per-source transcription failures in a batch SHOULD allow remaining independent sources to continue where possible.

Final batch result SHALL indicate error when at least one source failed.

90. Transcription logging content

Runtime logs SHOULD include at least:

- project/script version;
- run timestamp;
- selected model;
- language;
- device;
- compute type;
- source list;
- PyAV compatibility activation when applicable;
- per-source start/completion;
- segment count where available;
- detected language where available;
- duration where available;
- output paths;

- exceptions with traceback through logger exception handling;
- final return state.

91. Installer logging content

Installer logs SHOULD contain the same console information generated during the real modifying action, including:

- prerequisite state;
- Git-ignore additions/preservation;
- target paths;
- pip operations;
- validation output;
- resulting log path;
- final result.

92. Repository log isolation

Installer logs SHALL be placed under repository `logs/` rather than `.logs/`.

Transcription logs SHALL be placed under media/source `.logs/` rather than repository `logs/`.

This distinction is intentional:

<code>repo/logs/</code>	installer operations
<code>media/.logs/</code>	transcription operations

93. Privacy/data separation

The public/project package SHALL NOT include:

- user MP4/video sources;
- user audio sources;
- generated transcripts;
- generated `.logs/` runtime logs;
- generated installer logs;
- downloaded models;
- `.venv`;
- private exports;
- local caches.

94. Git model incident prevention

Because model payloads may be tens/hundreds/thousands of megabytes, `models/` SHALL be ignored by Git.

The installer SHALL ensure the protective rule is present to reduce the chance of accidentally pushing `model.bin` or related model assets.

95. ZIP/package policy

The delivered validation ZIP SHALL contain the complete updated repository source set, not only the scripts that changed most recently.

The ZIP SHALL contain every updated repository file listed in section 4.

It SHALL NOT contain runtime-only/private data listed in section 93.

96. ZIP root layout

The ZIP SHOULD expose the repository files directly at archive root rather than hiding them behind unrelated temporary directories.

97. ZIP integrity validation

Before delivery, the ZIP SHALL be tested for integrity.

The archive member list SHALL be inspected to confirm expected files and absence of excluded runtime directories.

98. Shell syntax validation

Every delivered `.sh` script SHALL pass:

```
bash -n
```

before delivery.

99. Python syntax validation

Every delivered `.py` script SHALL compile successfully with Python syntax/bytecode validation before delivery.

100. No-argument validation

Every user-facing script SHALL be tested with no arguments to verify help-only behavior.

101. Help validation

Help output SHALL be tested for all user-facing scripts.

`getModels` help SHALL be specifically checked for:

- complete 19-model list;

- list positioned before `EXAMPLES` ;
- multi-model syntax;
- `--force` explanation.

102. Changelog validation

Each script exposing `--changelog` SHALL be tested to verify current and historical entries are present.

103. Model parser validation

The model manager SHALL be tested for at least:

- one model;
- several model names after one `--model` ;
- a complete existing local model;
- an incomplete existing local model;
- `--force` simulation;
- `--force` execution behavior using an isolated test harness/mockdownloader;
- an invalid name mixed with valid names;
- custom `--models-dir`.

104. Transcription source parser validation

The transcription layer SHALL be tested for at least:

- one MP4;
- several MP4s;
- quoted `*.mp4` ;
- unquoted shell-expanded `*.mp4` ;
- filenames containing spaces;
- repeated `--source` ;
- several values after one `--source` ;
- direct Python backend parsing.

105. PyAV regression validation

The validation suite SHALL reproduce the PyAV 19 behavior where `metadata_errors` is rejected and verify that the local compatibility shim allows the simulated Faster-Whisper decode path to complete.

A mock may be used for this compatibility-unit validation when a full real model transcription is not appropriate inside the packaging runtime.

The package SHALL NOT falsely claim that a full real-world model/media transcription was executed when only a mock validation was performed.

106. Output-layout validation

Tests SHALL verify:

- timestamped Markdown remains beside the source;
- `.transcription/` contains plain Markdown/TXT;
- `.logs/` contains transcription logs;
- all output names contain `YYYYMMDD-HHMM-SS`;
- one run uses one shared timestamp;
- multi-directory batches create per-directory `.logs/` and `.transcription/` defaults.

107. Installer-log validation

Installer tests SHALL verify that real-action logging targets:

```
repo/logs/
```

with seconds present in the name.

Read-only/simulate test runs SHALL not create those logs.

108. .gitignore mutation validation

Tests SHALL verify:

- all original baseline lines remain;
- required missing lines are appended;
- already-present entries are not removed;
- duplicate baseline lines remain duplicated;
- `.logs/` is added/protected;
- `.log/` is not introduced;
- `logs/` is protected;
- `models/` is protected;
- `.venv/` is protected;
- `.transcription/` is protected.

109. .gitignore runtime idempotence

Repeated real installation SHALL not needlessly append another copy of an exact required rule that is already present.

This idempotence SHALL NOT be implemented by deduplicating historical duplicates; it only prevents the installer itself from appending a new duplicate of an already-present exact required line.

110. Requirements validation

Tests SHALL verify that:

- absent `requirements.txt` can be created by `install_pip.sh`;
- existing content is preserved;
- Faster-Whisper requirement is present after real bootstrap;
- `install.sh` consumes rather than recreates the dependency declaration.

111. Documentation requirements

The package SHALL include updated:

```
README.md
INSTALL.md
SPECIFICATIONS.md
EXAMPLES.md
CHANGELOG.md
```

Documentation SHALL describe current implemented behavior rather than still describing transcription/model management as future work.

112. EXAMPLES.md completeness

`EXAMPLES.md` SHALL be organized into sections by script.

It SHALL include examples for every supported argument, from simple to combined/complex workflows.

It SHALL include both shell wrappers and Python backends where Python business options exist.

113. INSTALL.md automation contract

`INSTALL.md` SHALL document a workflow where scripts create/manage their runtime pieces automatically.

Manual creation of `.venv`, `requirements.txt`, `.logs`, `.transcription` or required Git-ignore lines SHALL not be a required normal workflow step.

114. README.md current-state contract

`README.md` SHALL no longer say that the transcription layer is unimplemented.

It SHALL reflect the currently delivered installation, model-management and transcription layers.

115. CHANGELOG.md project history

`CHANGELOG.md` SHALL preserve the original installation history and add the current validation-candidate history, including at least:

- transcription introduction;

- V1.0.1 source/glob fix;
- PyAV 19 compatibility;
- new timestamped output layout;
- `.transcription/` and `.logs/`;
- installer logs;
- additive `.gitignore` management;
- multi-model downloads;
- model `--force`;
- documentation overhaul;
- removal of `WHY.md` from current package.

116. Current runtime baseline

The known successfully installed baseline recorded earlier in the project is:

```
Python      3.14.7
pip         26.2.1
faster-whisper 1.2.1
ctranslate2 4.8.2
PyAV       19.0.1
pip check   OK
WhisperModel import OK
```

This baseline is informative and SHALL not be treated as a hard requirement that every future machine use the exact same Python/pip version.

117. Python minimum

The installer SHALL require Python ≥ 3.9 unless a future Faster-Whisper runtime requirement raises the minimum.

118. Network/package behavior

The installation/model workflow SHALL use the configured Python/Hugging Face mechanisms rather than custom insecure download logic.

No HTTP-only dependency should be intentionally introduced by project scripts.

119. Model aliases

The project SHALL present model names exactly as exposed by the Faster-Whisper registry/help reference.

Aliases such as `large` and `turbo` SHALL remain selectable names when the runtime registry exposes them.

120. English-only model naming note

Model names ending in `.en` are English-oriented model variants.

For French transcription, multilingual names such as `tiny`, `base`, `small`, `medium`, `large-v*` or appropriate multilingual distil/turbo models are the relevant candidates.

The project SHALL not silently replace a requested model with another name.

121. Transcription output extension preservation

The current output base includes the original source filename including its extension.

Example source:

```
2011.mp4
```

produces names beginning:

```
2011.mp4.transcription...  
2011.mp4.transcript...
```

This convention is part of the current candidate and SHALL be documented consistently.

122. Source timestamp units

Internal segment timestamps SHALL be formatted to milliseconds:

```
HH:MM:SS.mmm
```

Generated **filenames** use wall-clock run timestamps to seconds:

```
YYYYMMDD-HHMM-SS
```

These are separate timestamp concepts and SHALL not be confused.

123. Runtime source ordering

Resolved sources SHOULD be processed in stable case-insensitive path order per resolved glob group while avoiding duplicate resolved paths.

124. Duplicate source suppression

If the same physical resolved source path is selected more than once in one invocation, the backend SHOULD process it only once.

125. Symlink/path resolution

Repository script paths SHALL be resolved to their physical script directory so the user may invoke scripts from a different current working directory.

This is required because media transcription is commonly launched from the media directory while scripts live in the project repository.

126. .venv interpreter discovery

Shell wrappers SHALL use:

```
<repo>/ .venv/bin/python
```

rather than whichever `python` happens to be first in the current user's shell `PATH`.

127. Backend discovery

`getModels.sh` SHALL locate `getModels.py` beside itself.

`transcribe.sh` SHALL locate `transcribe.py` beside itself.

Invocation from arbitrary media directories SHALL not break backend discovery.

128. Git operations boundary

The project scripts SHALL not automatically commit, push, force-push or rewrite Git history.

They MAY maintain `.gitignore` as specified.

Git commit/push decisions remain under user control.

129. Secrets policy

No credentials, access tokens, passwords or private authentication material SHALL be embedded in scripts or documentation.

130. Project-package exclusion policy

The validation ZIP SHALL exclude at least:

```
.venv/  
.VENV/  
venv/  
models/  
logs/  
.logs/  
.transcription/  
.zip/  
.old/  
.exports/  
.export/  
__pycache__/  
*.pyc  
source media
```

```
private transcripts
runtime logs
```

131. Requirements file in package

Even though `install_pip.sh` can create `requirements.txt` when absent, the delivered repository ZIP SHALL include the current `requirements.txt` as part of the complete project source package.

132. Package completeness

The user specifically requires a full-project ZIP for this update.

A partial ZIP containing only changed scripts SHALL be considered incomplete.

133. Validation before release

Successful automated checks on the delivered package do not replace user validation on the actual workstation/media/model set.

The package SHALL remain marked as a validation candidate until the user confirms expected real-world behavior.

134. Current acceptance checklist

The candidate is technically ready for user validation when all of the following are true:

1. every shell script passes syntax validation;
2. every Python script compiles;
3. no-argument help works;
4. script changelogs show the new candidate version;
5. getModels help contains the 19-model list before examples;
6. multi-model parsing works in shell and Python;
7. model force logic works in isolated tests;
8. quoted and unquoted transcription globs work;
9. filenames with spaces work;
10. PyAV 19 compatibility path is exercised in a test harness;
11. output naming uses `YYYYMMDD-HHMM-SS` ;
12. timestamped Markdown stays beside sources;
13. plain outputs go under `.transcription/` ;
14. transcription logs go under source `.logs/` ;
15. installer logs target repo `logs/` ;
16. `.gitignore` preservation/addition checks pass;
17. all expected documentation is present;
18. `WHY.md` is absent;
19. excluded runtime/private directories are absent from the ZIP;

20. ZIP integrity passes.

135. Changelog

v1.1.0-dev — 2026-10-09 18:32 CEST — Bruno DELNOZ

- CHANGED: Replaced obsolete installer-only specification with the exhaustive current project contract.
- ADDED: Complete installer, Git-ignore, model-management, transcription, output-layout, logging, compatibility, validation and packaging specifications.
- ADDED: Explicit regression requirements for globs, filenames with spaces and PyAV 19.
- ADDED: Multi-model downloads and `--force` model-replacement contract.
- ADDED: Source-local `.transcription/` and `.logs/` contract.
- ADDED: Repository-local installer `logs/` contract.
- ADDED: `YYYYMMDD-HHMM-SS` run timestamp contract.
- REMOVED FROM PACKAGE: `WHY.md`.
- STATUS: Validation candidate; not a release tag.

v1.0.0 — 2026-10-09 12:56 CEST — Bruno DELNOZ

- ADDED: Initial installer/bootstrap specifications and high-level future transcription boundary.